

2026 하반기 동아리 연계 WE-Meet 프로젝트

프로젝트 계획서

유니버스(Univ-Us)

대학생 학사·일정 개인 비서 AI 에이전트

팀 이름	Output()
팀 구성원	김하람, 김건희, 김영훈, 권세은, 윤지옥, 이예영, 정수빈, 정이슬
제출일	2026. 07. 19.
프로젝트 기간	2026. 07. 01. ~ 2026. 12. 31.

목차

목차.....	2
1. 프로젝트 개요.....	3
1.1 배경 및 목적.....	3
1.2 중요성 및 필요성.....	3
1.3 사용자 시나리오.....	4
2. 프로젝트 목표.....	5
2.1 구체적인 목표 설정.....	5
2.2 결과 및 산출물.....	5
2.3 목표 달성 판단 기준.....	6
3. 프로젝트 범위.....	6
3.1 목표별 추진 내용.....	6
3.2 주요 기능 및 특징.....	6
4. 프로젝트 방법론.....	10
4.1 연구 방법 및 절차.....	10
4.2 데이터 수집 및 분석.....	10
4.3 인공지능 모델 설계 및 개발 방법.....	11
4.4 주요 도구 및 기술.....	13
4.5 전문가 멘토링 계획.....	16
4.6 성과 지표 및 평가 방법.....	16
5. 일정 및 마일스톤.....	18
5.1 프로젝트 일정.....	18
5.2 마일스톤 및 목표.....	18
5.3 위험 관리 계획.....	19
6. 팀 구성 및 역할.....	20
6.1 팀원의 역할.....	20
6.2 팀원의 주요 업무.....	20
7. 관련 연구 및 문헌 조사.....	21
7.1 관련 문헌 및 선행연구 조사.....	21
7.2 기술 및 시장동향 조사.....	21
8. 결론.....	23
8.1 프로젝트 요약.....	23
8.2 기대효과.....	23
8.3 향후 계획.....	23
APPENDIX. 참고문헌 및 정보사이트.....	24
가. 참고문헌.....	24
나. 기술 문서·정보사이트.....	24

1. 프로젝트 개요

1.1 배경 및 목적

대학생의 학사 정보는 하나의 창구로 전달되지 않는다. 과제 마감은 LMS(e-Class)에, 시험·학사 일정은 공지사항에, 장학금·프로그램 공지는 학교 홈페이지와 각 사업단 게시판에, 성적·상담 안내는 이메일에 흩어져 있다. 학생은 이 정보들을 스스로 발견하고, 해석하고, 자신의 캘린더에 옮겨 적는 작업을 매 학기 반복하며, 이 과정에서 발생하는 누락은 과제 미제출, 수강신청·등록 기간 미확인, 장학금 신청 기회 상실과 같은 실질적인 손해로 이어진다.

기존의 캘린더 앱은 "학생이 직접 입력한 일정"만 관리할 수 있을 뿐, 흩어진 원천 문서를 읽고 일정을 만들어내는 단계는 여전히 학생의 몫이다. 본 프로젝트는 이 지점에 주목한다. 최근의 거대언어모델(LLM)은 비정형 문서에서 구조화된 정보를 추출하고, 사용자의 맥락에 맞춰 판단·요약하는 능력을 갖추었다. 이를 활용하면 "정보를 읽고 → 일정으로 변환하고 → 우선순위를 판단해 → 먼저 알려주는" 전 과정을 에이전트가 대신 수행할 수 있다.

이에 본 프로젝트는 e-Class(LMS)·공지사항·이메일 등 대학 생활의 원천 정보를 이해하여 과제·시험 마감을 캘린더에 자동 등록하고, 매일 아침 "오늘 할 일과 마감 임박 항목"을 브리핑하며, 장학금·공모전 등 학생 개인에게 맞는 기회를 선별해 알려주는 대학생 학사·일정 개인 비서 AI 에이전트 「유니버스(Univ-Us)」를 설계·구현하는 것을 목적으로 한다.

특히 본 프로젝트는 "모든 사람을 위한 범용 비서"가 아니라 "특정 사용자(대학생) × 특정 업무(학사·일정 관리)"로 문제를 좁혀, 제한된 개발 기간 내에 실제로 동작하고 체감 가치가 분명한 에이전트를 완성하는 전략을 취한다. 사용자와 업무 범위를 좁힐수록 데이터 형식(학사공지, 과제 정보)과 사용자 요구(마감 관리, 브리핑)가 명확해져 LLM 프롬프트·도구 설계의 정밀도를 높일 수 있기 때문이다.

1.2 중요성 및 필요성

- 실질적 수요: 마감·일정 관리는 전공을 불문하고 모든 대학생이 매일 겪는 문제로, 소수 기능만 완성되어도 즉시 체감 가치가 발생한다. 팀원 스스로가 최종 사용자이므로 요구사항 정의와 사용성 검증을 빠르게 반복할 수 있다.
- AI 에이전트 기술의 학습 가치: 본 과제는 문서 파싱(OCR), RAG(검색 증강 생성), 함수 호출(Function Calling) 기반 도구 사용, 스케줄링 등 최신 LLM 에이전트의 핵심 기술 요소를 고르게 포함한다.
- 국산 AI 생태계 활용: 한국어 학사 문서 처리라는 과제 특성에 맞춰 국산 한국어 특화 LLM(Upstage Solar)을 중심으로 설계함으로써, 한국어 환경에서의 LLM 응용 가능성을 검증한다.

- 확장성: 대학생 학사·일정 관리에서 검증된 "문서 이해 → 일정 변환 → 능동 브리핑"
파이프라인은 직장인 업무 비서, 사업단 프로젝트 관리 플랫폼 등 인접 도메인으로 확장 가능한 범용 구조이다.

1.3 사용자 시나리오

본 서비스가 대학생의 하루·한 학기에 어떻게 개입하는지를 대표 시나리오로 정리하면 다음과 같다. 각 시나리오는 3장에서 정의하는 각 기능과 직접 대응하며, 개발 우선순위 판단과 사용성 평가의 기준으로 활용한다.

시점	상황	에이전트의 동작
학기 시작 전	수강신청·등록금 납부 기간을 놓치기 쉽다	학사 공지에서 주요 일정을 추출해 캘린더에 등록하고 마감 전 알림(F1). 이수 내역을 바탕으로 남은 졸업요건을 제시(F2)
평일 아침	오늘 무엇을 먼저 해야 할지 판단이 어렵다	오늘의 강의·할 일·마감 임박 항목을 요약한 브리핑 제공(F10), 우선순위 상위 과제 제시(F7), 공강 시간에 학습 블록 배치(F8)
학기 중	결석 누적, 장학·공모전 공지 누락	결석 한도 도달 전 사전 경고(F3), 프로필과 자격요건이 맞는 장학·프로그램 공지만 선별 알림 및 신청서 초안 작성(F11~F13)
시험 기간	방대한 강의자료를 어디서부터 볼지 막막하다	강의자료 요약과 예상 문제 생성(F4), 시험일까지 역산한 공부 일정 추천(F5)
팀 프로젝트	팀원 일정 조율과 역할 관리에 시간이 소모된다	팀원 공강 교집합 기반 회의 시간 추천 및 역할·마감 추적(F16)

위 시나리오의 공통점은 학생이 "정보를 찾아 나서는" 대신 에이전트가 "필요한 시점에 먼저 알려주는" 구조라는 점이다. 본 프로젝트는 이 능동성(proactiveness)을 제품의 핵심 가치로 삼는다.

2. 프로젝트 목표

2.1 구체적인 목표 설정

프로젝트의 최종 목표는 "대학생이 학사 일정을 직접 관리하지 않아도 되는 상태"를 만드는 것이며, 이를 다음 4개의 구체적 목표로 분해한다.

구분	목표	세부 내용
G1	문서 기반 일정 자동화	e-Class(LMS)의 과제 정보와 공지사항·이메일에서 과제·시험·학사 일정을 추출하여 사용자 캘린더에 자동 등록. 추출 정확도(마감일 기준) 90% 이상 달성.
G2	능동적 브리핑·알림	매일 아침 정해진 시각에 "오늘 할 일 + 마감 임박 + 권장 학습 계획"을 요약 브리핑으로 제공하고, 결석 한도·학사 마감 등 위험 신호를 사전 경고.
G3	개인 맞춤 판단·추천	마감일·성적 비중·예상 소요시간 기반의 과제 우선순위 산정, 공강 시간 기반 학습 블록 배치, 저장된 개인 정보(학년·이수학점 등)와 장학·대외활동 공지의 자동 매칭.
G4	학습 지원 RAG 구축	업로드한 강의자료 기반의 요약·시험 예상 문제 생성 기능을 RAG 파이프라인으로 구현하여, 일정 관리를 넘어 학습 단계까지 지원.

2.2 결과 및 산출물

- 동작하는 웹 서비스 1 식: Next.js 기반 웹 인터페이스 + FastAPI 기반 에이전트 백엔드(데모 가능한 배포 환경 포함)
- AI 파이프라인 3 종: ① 문서 → 일정 구조화 추출 파이프라인, ② 아침 브리핑 생성 파이프라인, ③ 강의자료 RAG(요약·예상문제) 파이프라인
- 데이터베이스 스키마 및 API 명세서(FastAPI 자동 문서 포함)
- 성능 평가 보고서: 자체 구축 테스트셋 기반 추출 정확도·응답 품질 평가 및 모델 비교 실험 결과

산출물의 완성 기준은 "문서상 설계"가 아니라 "실제로 동작하는 데모"이다. 각 산출물은 팀 저장소(GitHub)에 코드·문서와 함께 축적하며, 배포된 데모 환경에서 상시 시연이 가능한 상태를 유지하는 것을 원칙으로 한다.

2.3 목표 달성 판단 기준

구분	판단 기준
필수 달성(Must)	공지·과제 기반 일정 자동 등록, 아침 브리핑, 과제 우선순위 추천, 자연어 일정 등록·질의가 통합 데모에서 오류 없이 동작할 것
목표 달성(Should)	졸업요건 트래커, 출결 경고, 학습 블록 배치, 강의자료 RAG, 장학·대외활동 매칭 알림까지 동작하며 성과 지표(4.6)를 충족할 것
도전 달성(Could)	팀플 조율, 사업단 프로젝트 관리 플랫폼까지 포함한 전 기능 통합 및 시범 사용자 운영

3. 프로젝트 범위

3.1 목표별 추진 내용

본 프로젝트는 기획 단계에서 정의한 7개 기능군, 15개 기능 전체를 핵심 기능으로 구현하는 것을 범위로 한다. 각 기능은 2장에서 정의한 4개 목표(G1~G4)에 다음과 같이 대응된다.

목표	내용	대응 기능
G1	문서 기반 일정 자동화	F1 학사일정 자동 등록, F6 과제 마감 자동 등록
G2	능동적 브리핑·알림	F3 출결·학사경고 예방, F10 아침 브리핑, F11~F13 장학·대외활동·사업단 공지 알림
G3	개인 맞춤 판단·추천	F2 졸업요건 트래커, F5 시험 공부 일정, F7 우선순위, F8 학습 블록 배치, F9 자연어 질의, F16 팀플 조율
G4	학습 지원 RAG 구축	F4 강의자료 요약·시험 예상 문제 생성
확장	플랫폼화	F17 사업단 프로젝트 관리 플랫폼

3.2 주요 기능 및 특징

가. 학사 관련

ID	기능	내용
F1	학사 일정 자동 등록·알림	학기 등록 기간, 등록금 납부 기간, 수강 신청 기간 등 주요 학사 일정을 수집해 개인 캘린더에 추가하고 시점별 알림을 제공한다.
F2	졸업요건·학점 트래커	이수 내역을 입력하면 남은 전공·교양 요건과 부족분을 계산해 알려준다. 요건 계산은 정확성 보장을 위해 LLM이 아닌 규칙 기반 코드로 구현한다.

나. 강의 관련

ID	기능	내용
F3	출결·학사경고 예방 알림	과목별 결석 한도(예: 1/4 규정)를 추적해 위험 수위에 도달하기 전에 미리 경고한다.
F4	강의자료 요약·시험 예상 문제 생성	업로드한 강의 PDF·필기를 요약하고 시험 대비 예상 문제를 자동 생성한다. RAG 파이프라인으로 구현해 근거(출처 페이지)를 함께 제시한다.
F5	시험 공부 일정 자동 추천	강의자료의 양과 난이도, 시험 일자를 분석해 역산형 공부 일정을 추천한다(F8 학습 블록 배치와 연동).

다. 과제·일정 관련

ID	기능	내용
F6	과제 마감 자동 등록	e-Class(LMS)에 로그인해 본인 수강 과목의 과제와 마감 일정을 자동으로 수집하고, 마감기한을 개인 캘린더에 등록해 알림을 설정한다. 수집은 Playwright 기반 브라우저 자동화로 구현하며, 최초 1회 SSO 로그인 시 저장되는 신뢰 기기 쿠키(약 1년, 2차 인증 면제)와 세션을 이용해 이후에는 작업 스케줄러로 창 없이 주기적으로 무인 수집한다. 과제 설명·종료일시·제출/채점 상태, 공지사항·자료실 글, 강의자료 파일을 수집 대상으로 하며, 동영상 본체·퀴즈·출결과 타인이 작성한 글은 수집하지 않는다. 요청 간격 제한(1.5초)·동시성 1, 문서 확장자만 수집, 수집물 외부 미배포 등 학교 저작권·개인정보 기준을 준수한다.
F7	과제 우선순위 확인·배치	마감일 + 성적 반영 비중 + 예상 소요시간을 종합해 과제 우선순위를 매기고 브리핑에 반영한다.
F8	공강 기반 학습 플랜 자동 배치	시간표의 빈 시간과 남은 마감을 보고 "화 14-16시 A과제, 목 저녁 B 쪽지시험 공부"처럼 학습 블록을 캘린더에 자동으로 배치한다.

ID	기능	내용
F9	자연어 일정 등록·질의	"다음 주 목요일 발표 리허설 저녁 8시에 넣어줘", "이번 주 마감 뭐 남았어?" 같은 대화형 일정 조작·조회를 Function Calling으로 지원한다.
F10	아침 브리핑	매일 아침 정해진 시각에 오늘의 강의·할 일·마감 임박(D-3 이내) 항목과 권장 학습 계획을 요약해 대시보드·알림으로 제공한다.

라. 장학 관련

ID	기능	내용
F11	장학 공지 매칭·신청서 자동 작성	대학교 홈페이지에 장학 공지가 올라오면 내용을 확인해, 미리 저장된 내 정보(예: 2학년, 이번 학기 18학점 이상 이수)와 장학 조건이 일치하면 장학금 신청서를 자동으로 작성한다. 최종 제출은 사용자가 확인·승인 후 진행하는 반자동 방식으로 안전성을 확보한다.

마. 대외 활동 관련

ID	기능	내용
F12	맞춤 기회 알림	공모전, 교내 프로그램 등 저장된 관심사에 맞는 기회를 선별해 알린다.
F13	사업단 공지 통합 알림	학교 홈페이지 외에 학교 내 여러 사업단 게시판의 공지까지 수집 범위에 포함해 놓치기 쉬운 정보를 통합 제공한다.

바. 생활·협업(기타)

ID	기능	내용
F16	팀플 일정 조율	팀원들의 빈 시간 교집합을 찾아 회의 시간을 추천하고, 역할·마감 분담을 추적한다.

사. 사업단 프로젝트 관련

ID	기능	내용
F17	사업단 프로젝트 관리 플랫폼	사업단에 참여하는 멘토들의 프로필을 등록해 대학생들이 멘토를 찾아볼 수 있도록 하고, 멘토·팀원들의 과제 신청·등록, 과제 선정과 수행 중·결과 단계에서 진행되는 보고서 업로드, 관리자의 승인이 필요한 사항에 대한 승인 절차까지 이루어지도록 구현한다. 본 과제의 문서 처리·알림·승인 워크플로를 재사용한다.

제품 특징

- "읽고 → 판단하고 → 대신 하는" 에이전트: 단순 일정 저장 앱과 달리 원천 문서를 직접 이해해 일정을 만들어내고(F1·F6), 우선순위를 판단하며(F7), 매일 능동적으로 브리핑한다(F10). 사용자의 입력 부담 최소화가 제품의 핵심 차별점이다.
- 한국어 학사 문서 특화: 한국어 특화 LLM 과 문서 파싱 기술을 결합해, 한글 공지·과제·행정 용어가 섞인 실제 학사 문서에서의 추출 품질을 우선한다.
- 신뢰 우선 설계: 캘린더 등록·장학금 신청서 제출 등 실제 행동이 수반되는 기능은 에이전트가 초안을 만들고 사용자가 확인·승인하는 Human-in-the-loop 방식을 기본으로 한다.
- 개인정보 최소 수집: 서비스에 필요한 최소 정보(시간표, 이수학점, 관심사 등)만 동의 하에 저장하고, 원문 문서는 처리 후 사용자가 삭제할 수 있게 한다.

구현 단계화 원칙

15개 기능 전체가 최종 산출물의 핵심 범위이나, 5개월의 개발 기간을 고려해 기능 간 의존성과 데모 가치에 따라 구현 순서를 단계화한다. 파이프라인의 뼈대가 되는 기능(F1·F6·F9·F10)을 1단계로 완성해 이후 기능들이 공유하는 기반(문서 추출, 캘린더 연동, 대화 인터페이스, 스케줄러)을 확보하고, 판단·추천 기능(F2·F3·F5·F7·F8)과 수집·매칭 기능(F11~F13), RAG(F4)를 2단계, 조합·플랫폼 성격의 기능(F16·F17)을 3단계로 구현한다. 세부 일정은 5장에 기술한다.

4. 프로젝트 방법론

4.1 연구 방법 및 절차

짧은 주기로 "동작하는 결과물"을 반복 완성하는 애자일 방식을 채택한다. 2주 단위 스프린트로 개발을 진행하며, 각 스프린트 종료 시 팀 회의에서 데모·회고를 수행하고 기업 멘토링에서 받은 피드백을 다음 스프린트 백로그에 반영한다. 전체 절차는 다음과 같다.

- 1 단계 — 요구분석·설계(7 월): 사용자 시나리오 정의, 데이터 소스(e-Class 과제·공지·이메일) 샘플 수집, DB 스키마·API·에이전트 아키텍처 설계, 개발 환경 구축
- 2 단계 — 기반 기능 구현(8~9 월): F1·F6(일정 자동 수집·등록) → F10(아침 브리핑) → F7(우선순위) → F9(자연어 질의) 순으로 서비스 뼈대 완성
- 3 단계 — 판단·수집 기능 구현(10~11 월): F2·F3·F5·F8(판단·추천), F11~F13(공지 수집·매칭), F4(강의자료 RAG)를 병렬 개발, 통합 테스트 및 성능 평가 병행
- 4 단계 — 조합·플랫폼 기능 및 마무리(11~12 월): F16(팀플 조율), F17(사업단 플랫폼) 구현, 사용성 개선, 평가 실험 완료, 결과보고서·발표 자료 작성

과제·마감 수집(F6)은 e-Class(LMS) 자동 로그인 크롤링으로 구현하며, 전용 도구(eclass_agent)로 최초 1회 로그인 후 세션·신뢰 기기 쿠키를 이용한 주기적 무인 수집을 검증하였다. 인증 정보는 Windows 암호화 저장(완전 무인) 또는 미저장(반자동)을 선택할 수 있게 하고, 학교 저작권·이용약관 기준(요청 간격 제한, 문서만 수집, 미배포)을 준수한다.

4.2 데이터 수집 및 분석

데이터	수집 방법	활용
학사 공지·사업단 공지	학교 홈페이지·사업단 게시판 공개 공지 수집	F1·F11~F13 수집·매칭 실험
이메일	팀원 계정의 학사 관련 메일(Gmail API, 본인 동의)	F6 이메일 파싱 실험
시간표·이수내역	팀원 본인 데이터 직접 입력	F2·F7·F8 로직 검증
강의자료	팀원 수강 과목 자료(저작권 고려, 개인 학습 범위 내 사용)	F4 RAG 색인·평가

수집한 문서 중 약 50건 이상을 골라 마감일·과제명 등 정답을 수작업으로 라벨링한 평가용 테스트셋을 구축하고, 이를 추출 정확도 측정과 프롬프트 개선의 기준으로 삼는다. 개인정보가 포함된 데이터는 팀원 본인 동의 하에서만 사용하며 외부에 공개하지 않는다.

4.3 인공지능 모델 설계 및 개발 방법

가. 후보 모델 비교

주 LLM 선정을 위해 상용 API 모델과 오픈소스 모델을 폭넓게 후보로 검토하였다. 본 과제의 요구사항은 ① 한국어 학사 문서(공지·과제·행정 용어)에 대한 이해력, ② 일정 추출을 위한 구조화 출력·Function Calling 지원, ③ 문서를 대량 반복 처리하는 사용 패턴을 감당할 수 있는 낮은 토큰 단가, ④ 문서 본문과 대화 이력을 함께 다룰 수 있는 컨텍스트 길이의 4가지로 정리된다. 아래 표는 2026년 상반기 공시 가격 기준(1M 토큰당 USD, 입력/출력, 변동 가능)의 후보 비교이다.

후보 모델	한국어	가격(입력/출력)	컨텍스트	비고
Upstage Solar Pro 3	특화(국산)	\$0.15 / \$0.60	128K	Function Calling·구조화 출력 지원, MoE 구조로 비용 효율 높음
GPT-5 계열(상위)	우수	약 \$2.5 / \$15	272K~	범용 성능 최상위권이나 단가 높음
GPT 경량(mini급)	양호	약 \$0.75 / \$4.5	272K~	경량 모델 중 균형형, Solar 대비 고단가
Claude Sonnet 계열	우수	약 \$3 / \$15	200K~1M	긴 문서·추론 강점, 단가 높음
Claude Haiku 계열	양호	약 \$1 / \$5	200K	경량·저지연, 한국어 특화는 아님
Gemini Pro 계열	우수	약 \$2 / \$12	1M	초장문 컨텍스트 강점
Gemini Flash 계열	양호	약 \$0.5 / \$3	1M	저가·고속, 대량 처리 적합
오픈소스(EXAONE, Qwen, Llama 등)	모델별 상이	GPU 비용	모델별 상이	API 비용은 없으나 서버용 GPU 인프라·운영 부담이 학생 프로젝트에 과중

비교 결과, 프런티어급 상용 모델(GPT 상위·Claude Sonnet·Gemini Pro)은 범용 성능이 뛰어나지만 본 과제의 대량 문서 파싱 패턴에서는 토큰 단가가 과도하고, 오픈소스 자체 서버는 GPU 인프라 확보와 운영이 5개월 학생 프로젝트의 부담 범위를 넘는다. Solar Pro 3는 한국어 특화, Function Calling·구조화 출력 지원, 경량 상용 모델 중 가장 낮은 단가라는 조건은 충족하나, 128K 컨텍스트는 후보 중 가

장 짧은 수준이다.

본 과제가 다루는 문서는 e-Class 과제·공지·이메일 본문과 강의자료로, 개별 문서의 길이가 짧아(대개 수천 토큰 이내) 자연어 질의(F9)에서 누적되는 대화 이력을 함께 고려하더라도 128K 컨텍스트에 충분히 들어온다. 따라서 컨텍스트 길이 요구사항은 문제가 되지 않으며, 한국어 특화·구조화 출력 지원·경량 상용 모델 중 가장 낮은 단가라는 조건을 함께 고려해 Solar Pro 3를 주 모델로 최종 선정한다.

나. 모델 구성 및 기능별 운용 전략

단일 모델에 모든 작업을 맡기지 않고, 작업 특성에 따라 모델을 나누어 운용한다.

구성 요소	모델	담당 작업
주 LLM	Upstage Solar Pro 3	일정 추출(F1-F6), 브리핑 생성(F10), 대화형 질의(F9), 요약·예상 문제 생성(F4), 공지 매칭 판단(F11~F13)
문서 파싱	Upstage Document Parse	PDF·이미지 형태의 공지·강의자료의 텍스트·표 구조 추출(OCR). Solar Pro 3가 텍스트 전용 모델이므로 이미지 계열 입력의 전처리를 전담
임베딩	Upstage Solar Embedding	강의자료·공지 청크의 벡터화(F4 RAG, F11 매칭 검색)
규칙 기반 로직	(비 LLM) Python 코드	졸업요건 계산(F2), 결석 한도 추적(F3), 우선 순위 점수식(F7) 등 정확성이 요구되는 결정적 계산
예비(fallback) 모델	상용 상위 모델(비교 실험으로 선정)	테스트셋 평가에서 Solar의 정확도가 목표에 미달하는 특정 작업에 한해 선택적 대체 적용

LLM 호출부는 LangChain의 모델 어댑터 계층으로 추상화하여 특정 모델에 종속되지 않는 구조로 설계한다. Solar Pro 3와 Gemini 계열은 OpenAI 호환 엔드포인트를 지원하므로 동일 인터페이스로 호출 가능하나, 예비 모델 후보인 Claude 계열은 Anthropic 고유 API 스펙을 사용하므로 별도 어댑터를 구현한다. 이를 통해 기능별 모델 교체는 설정 변경 + 필요 시 어댑터 선택만으로 가능하도록 하며, 동일 테스트셋에서 후보 모델 간 정확도·비용을 비교하는 실험(4.6)을 수행해 선정 근거를 정량적으로 검증한다. 판단이 필요한 작업은 LLM에, 결정적 계산은 규칙 기반 코드에 맡기는 역할 분리를 통해 정확성과 유연성을 동시에 확보한다.

다. 핵심 파이프라인 설계

파이프라인	설계
① 문서 → 일정 추출 (F1-F6)	e-Class 크롤링으로 수집한 과제·마감 정보와 학사 공지 텍스트(이미지·PDF는 Document Parse로 텍스트화) → Solar Pro 3에 일정 추출 프롬프트 + JSON 스키마 지정 → {과목, 유형(과제/시험/학사), 제목, 마감일시, 출처} 형태로 구조화 추출 → 검증 규칙(날짜 유효성, 중복 검사) 통과 후 사용자 확인 → Google Calendar 등록 및 알림 설정
② 아침 브리핑 생성 (F10)	스케줄러(APScheduler)가 매일 아침 실행 → DB에서 오늘 일정·마감 임박 항목·우선순위 점수(F7)·학습 블록(F8) 조회 → Solar Pro 3가 개인화 브리핑 텍스트 생성 → 대시보드·알림 발송
③ 강의자료 RAG (F4)	강의 PDF를 청크 분할 → Solar Embedding으로 벡터화 → pgvector에 저장 → 질의 시 유사 청크 검색 → Solar Pro 3가 근거 기반 요약·예상 문제 생성(출처 페이지 표기)
④ 대화형 에이전트 (F9)	사용자 자연어 입력 → Solar Pro 3 Function Calling으로 의도 분류 및 도구 선택(일정 등록/조회/수정, 우선순위 질의, 요건 조회 등) → 도구 실행 결과를 자연어로 응답. 일정 변경 등 쓰기 작업은 실행 전 사용자 확인을 거친다.
⑤ 공지 수집·매칭 (F11~F13)	스케줄러가 학교·사업단 공지 게시판을 주기 수집 → 신규 공지 의 자격 요건을 Solar Pro 3로 구조화 추출 → 저장된 사용자 프로필(학년·학점·전공·관심사)과 규칙 매칭 → 해당자에게 알림, 장학(F11)은 신청서 초안 자동 작성 후 사용자 승인 대기
Playwright (e-Class 자동 수집)	https://playwright.dev/python

라. 품질 관리

- 프롬프트 버전 관리: 추출·브리핑 프롬프트를 코드와 함께 버전 관리하고, 테스트셋 정확도가 개선된 경우에만 반영한다.
- 환각 억제: 추출 결과는 원문 근거(출처 문장)를 함께 반환하도록 하여 사용자가 검증할 수 있게 하고, RAG 응답에는 출처 페이지를 표기한다.
- 실패 처리: 추출 신뢰도가 낮은 항목은 자동 등록하지 않고 "확인 필요" 목록으로 분리해 사용자 검토를 요청한다.
- 비용 관리: 기능별 토큰 사용량을 로깅해 월 예상 비용을 모니터링하고, 반복 호출 구간에는 프롬프트 캐싱·결과 캐싱을 적용한다.

4.4 주요 도구 및 기술

가. 시스템 아키텍처 개요

시스템은 4개 계층으로 구성된다. ① 프레젠테이션 계층(Next.js 웹)은 대시보드·브리핑·대화 UI와 문서 업로드를 담당하고, ② 애플리케이션 계층(FastAPI)은 REST API와 에이전트 오케스트레이션(도구 호출 라우팅)을 수행하며, ③ AI 계층은 Solar Pro 3·Document Parse·Solar Embedding 호출과 RAG 검색을 담당하고, ④ 데이터 계층(PostgreSQL + pgvector)은 사용자·일정·문서·벡터 데이터를 저장한다. 스케줄러(APScheduler)는 애플리케이션 계층에서 아침 브리핑(F10)·공지 수집(F11~F13) 등 정기 작업을 실행하며, 외부 연동(Google Calendar·Gmail)은 OAuth 2.0 기반으로, e-Class 자동 수집은 별도 크롤링 모듈로 애플리케이션 계층에서 관리한다.

나. 프론트엔드

- Next.js(React) + TypeScript: App Router 기반으로 대시보드(오늘 일정·브리핑·우선순위), 문서 업로드, 대화 인터페이스, 설정(프로필·관심사) 화면을 구성한다.
- 컴포넌트 라이브러리(shadcn/ui 등) + Tailwind CSS: 일관된 UI를 빠르게 구축해 프론트엔드 작업량을 최소화하고 백엔드·AI 개발에 역량을 집중한다.
- 데이터 페칭·상태 관리(React Query 등): 브리핑·일정 데이터의 캐싱과 갱신을 관리하고, 대화 응답은 스트리밍으로 표시해 체감 응답 속도를 높인다.

다. 백엔드

- Python + FastAPI: 비동기(async) 처리로 LLM API 호출 대기 시간을 효율적으로 다루고, 자동 생성되는 OpenAPI 문서를 API 명세서 산출물로 활용한다.
- Pydantic 스키마 검증: LLM의 구조화 출력(JSON)을 Pydantic 모델로 검증해, 형식 오류·누락 필드를 파이프라인 단계에서 차단한다. 일정 추출 품질 관리(4.3 라)의 기술적 기반이다.
- 인증·보안: JWT 기반 세션 관리, 외부 서비스 토큰(Google OAuth)의 암호화 저장, API 요청 검증.

라. 데이터베이스 설계

주요 테이블	내용
users / profiles	계정, 학년·전공·이수학점·관심사 등 매칭용 프로필(F11~F13)
courses / timetable	수강 과목, 시간표(공강 계산의 기준 데이터, F8·F16)
events / tasks	일정·과제(마감, 비중, 예상 소요시간, 우선순위 점수, 출처 문서 참조)
documents / chunks	업로드 문서 메타데이터, RAG용 청크와 임베딩 벡터(pgvector, F4)
notices	수집한 학사·장학·사업단 공지와 구조화 추출된 자격 요건(F1·F11~F13)

주요 테이블	내용
attendance / requirements	과목별 출결 기록(F3), 졸업요건 규칙·이수 현황(F2)
mentors / projects / reports	사업단 플랫폼(F17)용 멘토 프로필, 과제 신청, 보고서·승인 이력

마. AI-RAG 계층

- LangChain: LLM 호출 추상화, 도구(Function) 정의·라우팅, RAG 체인 구성에 사용한다. 특정 프레임워크 종속을 줄이기 위해 핵심 프롬프트·스키마는 자체 모듈로 분리 관리한다.
- 청크 전략: 강의자료는 페이지·소제목 단위로 분할하고 출처(파일명·페이지)를 메타데이터로 저장해, 응답 시 근거 표기가 가능하도록 한다.
- 검색: pgvector 코사인 유사도 기반 top-k 검색을 기본으로 하고, 품질 평가 결과에 따라 키워드 검색 병행(하이브리드)을 적용한다.

바. 외부 연동

- Google Calendar API(OAuth 2.0): 일정 등록·수정·알림 설정. 최소 권한 범위(scope)만 요청한다.
- Gmail API: 학사 관련 메일 수집(라벨·발신자 필터 기반). 본문은 일정 추출 후 원문 보관 여부를 사용자가 선택한다.
- 알림 채널: 1 차는 웹 푸시·이메일로 구현하고, 카카오톡 알림 연동은 고도화 단계에서 검토한다.

사. 스케줄러·배치

- APScheduler: 아침 브리핑(매일 지정 시각), 공지 수집(수 시간 간격), 마감 임박 알림(일 단위) 등 정기 작업을 크론 방식으로 실행한다. 작업 실패 시 재시도와 로깅을 포함한다.

아. 개발·배포 환경

- GitHub: 브랜치 전략(main/develop/feature), Pull Request 코드 리뷰, Issue 기반 백로그 관리.
- Docker: 백엔드·DB 를 컨테이너화해 팀원 간 개발 환경을 통일하고 배포를 단순화한다.
- 배포: 프론트엔드는 Vercel, 백엔드·DB 는 클라우드 VM(Docker Compose)에 배포해 상시 데모 가능한 환경을 유지한다.
- 품질 도구: pytest(백엔드 테스트), ruff/ESLint(정적 분석), GitHub Actions(CI) 적용.

자. 보안·개인정보 처리

- 최소 수집 원칙: 기능 수행에 필요한 정보만 동의 기반으로 수집하고, 수집 항목을 화면에 명시한다.
- 보관·삭제: 외부 서비스 토큰과 민감 정보는 암호화 저장하고, 사용자가 원문 문서·계정 데이터를 직접 삭제할 수 있는 기능을 제공한다.
- LLM 전송 최소화: 개인 식별 정보는 프롬프트에 포함하지 않는 것을 원칙으로 하고, 필요한 경우 항목 단위로 최소화해 전달한다.

4.5 전문가 멘토링 계획

멘토와의 멘토링을 프로젝트 기간 중 3회 이상 실시하며, 이 중 2회 이상은 오프라인으로 진행한다.

회차	시기(안)	방식	주제(안)
1차	7월 16일	오프라인	주제 선정 및 구체화
2차	9월 ~ 10월	오프라인	기반 기능 데모 피드백, RAG 품질·프롬프트 개선 자문
3차	11월 ~ 12월 초	온라인 또는 오프라인	최종 완성도 점검, 평가 실험 설계·결과 해석, 발표 준비

프로젝트 진행 현황 점검 및 방향성 체크, 팀원 질의응답

1. 프로젝트 기술 자문

- 프로젝트를 진행하면서 필요한 기술에 대한 자문 및 디버그 조언을 진행한다.

2. 프로젝트 회의 참여

- 팀원의 요청시 멘토는 회의에 참석 할 수 있다.

3. 사업단과의 연락

- 프로젝트 변경 시 사업단과 이슈를 해결한다.

팀원은 코딩능력 습득, 인공지능 기술 동향이해를 통한 프로젝트 산출물을 도출한다.

- 깃허브 저장소 소스코드 일정 및 버전관리
- 팀원 역할 분담을 통한 과제 솔루션 제출

팀원들 간의 협업하고 피드백을 받을 수 있으며 실습을 통해 GitHub와 같은 버전 관리도구를 익히게 된다. 이로 인해 실제 개발 환경에서의 실무역량이 향상될 것으로 기대 된다.

4.6 성과 지표 및 평가 방법

지표	측정 방법	목표
일정 추출 정확도	자체 테스트셋(50건 이상)에서 마감일·과제명 추출의 정확도 측정	마감일 기준 90% 이상
브리핑 품질	팀원·시범 사용자 만족도 평가(5점 척도) 및 오류 신고 건수	평균 4.0 이상
RAG 응답 근거성	예상 문제·요약의 출처 페이지 일치 여부 표본 검사	근거 표기율 95% 이상
공지 매칭 정밀도	장학·프로그램 공지 매칭 결과의 자격 요건 일치 여부 검수	오매칭률 10% 이하
응답 시간	문서 추출·대화 응답의 평균 처리 시간 측정	대화 응답 5초 이내
모델 비교 실험	동일 테스트셋에서 주 모델과 대체 모델의 정확도·비용 비교	선정 근거 정량화
사용성 검증	팀원 외 시범 사용자 5인 이상 2주 사용 후 인터뷰	개선 항목 도출·반영

5. 일정 및 마일스톤

5.1 프로젝트 일정

전체 수행 기간(2026.07.01~12.31)을 4개 단계, 12개 스프린트(2주 단위)로 구성한다. 기능 간 의존성을 고려해 기반 기능을 먼저 완성하고, 이후 판단·수집·조합 기능을 순차 개발한다.

기간	주요 활동	산출물
7월	요구분석, 사용자 시나리오·아키텍처 설계, DB 스키마 설계, 계획서 제출, 개발 환경 (GitHub-Docker) 구축, 샘플 문서 수집	프로젝트 계획서, 설계 문서, 개발 환경
8월	F6·F1 수집·추출 파이프라인 구현(e-Class 자동 크롤링, JSON 추출, 캘린더 등록), 평가 테스트셋 구축, 1차 멘토링(오프라인)	일정 추출 파이프라인 v1, 테스트셋 50건
9월	F10 아침 브리핑(스케줄러), F7 우선순위 산정, F9 자연어 질의(Function Calling) 구현 → 기반 통합 데모	통합 데모 v1(F1·F6·F7·F9·F10)
10월	F2 졸업요건 트래커, F3 출결 경고, F5·F8 학습 일정 배치, F11~F13 공지 수집·매칭 구현, 2차 멘토링(오프라인)	판단·수집 기능 v1
11월	F4 강의자료 RAG 구현, F16 팀플 조율 구현, 통합 테스트, 시범 사용자 평가(5인 이상)	RAG 파이프라인, 베타 버전, 평가 데이터
12월	F17 사업단 플랫폼 구현, 성능 평가·모델 비교 실험 완료, 고도화·버그 수정, 3차 멘토링, 결과보고서 작성·제출(12.18~12.31)	최종 버전, 평가 보고서, 결과보고서, 발표 자료

5.2 마일스톤 및 목표

구분	시점	달성 기준
M1	7월 19일	프로젝트 계획서 제출 완료, 개발 환경·저장소·설계 문서 확보
M2	8월 말	e-Class 과제·마감 → 캘린더 자동 등록(F6·F1) 동작, 테스트셋 기반 추출 정확도 1차 측정
M3	9월 말	기반 기능 통합 데모 가능(F1·F6·F7·F9·F10), 배포 환경에서 상시 시연 가능
M4	11월 말	판단·수집·RAG 기능 포함 베타 완성(F2~F5·F8·F11~F13·F16), 시범 사

구분	시점	달성 기준
		용자 평가 완료
M5	12월 말	F17 포함 전 기능 통합, 성과 지표 달성 확인, 결과보고서·발표 자료 제출

5.3 위험 관리 계획

일정·품질 위험을 사전에 식별하고 대응 방안을 마련해 두어, 문제 발생 시 프로젝트 전체가 지연되지 않도록 관리한다.

위험 요소	영향	대응 방안
LMS 자동 로그인 크롤링의 약관·세션 안정성	F6 수집 지연	eclass_agent의 준수 규칙(요청 간격 1.5초·동시성 1, 문서만 수집, 미배포) 적용, 세션 만료 시 신뢰 기기 쿠키로 자동 재로그인·실패 시 재시도, 비밀번호는 DPAPI 암호화 저장 또는 미저장(반자동) 방식 선택
한국어 문서 추출 정확도 미달	핵심 지표 미달	프롬프트·스키마 개선 반복, 신뢰도 낮은 항목은 "확인 필요"로 분리, 필요 시 대체 모델 부분 적용(4.3나)
기능 범위 과다로 인한 일정 지연	M4·M5 지연	기능을 필수·목표·도전(2.3)으로 계층화해 관리하고, 스프린트 회고에서 범위를 조정
외부 API 정책·요금 변경	비용·연동 실패	LLM 호출부 추상화로 모델 교체 용이성 확보, 토큰 사용량 모니터링 및 캐싱 적용
개인정보·저작권 이슈	데이터 활용 제한	팀원 동의 데이터만 사용, 강의자료는 개인 학습 범위 내 활용, 최소 수집·삭제 기능 제공
팀원 학사 일정(시험·과제) 집중기	개발 속도 저하	중간·기말고사 기간을 저부하 스프린트로 편성하고 완충 기간을 일정에 반영

6. 팀 구성 및 역할

6.1 팀원의 역할

역할	팀원	담당
팀장 / 기획	김하람	프로젝트 총괄, FastAPI 백엔드, LLM 파이프라인(일정 추출·브리핑) 개발, 사업단·멘토 커뮤니케이션, 일정·산출물 관리
AI·데이터	김건희, 윤지옥, 정수빈	RAG 파이프라인(F4), 프롬프트 엔지니어링, 공지 수집·매칭(F11~F13), 테스트셋 구축·모델 비교 실험
백엔드	김영훈, 권세은, 정이슬	Next.js 대시보드·브리핑·대화 UI, 문서 업로드 흐름, 캘린더 OAuth 연동 프론트, 사용성 개선
프론트·인프라	김하람, 이예영	DB·배포 환경(Docker·CI), 스케줄러 운영, 통합 테스트, 기술 문서화

6.2 팀원의 주요 업무

가. 공통 업무

- 2 주 단위 스프린트 참여: 계획 수립 → 개발 → 데모·회고
- 주 1 회 정기 팀 회의 진행 및 회의록 작성
- 기업 멘토링 참석 및 피드백의 백로그 반영
- 담당 기능의 테스트 코드 작성, 코드 리뷰(Pull Request) 참여, 기술 문서 작성

나. 협업 방식

항목	운영 방식
일정 관리	GitHub Issue·Project 보드로 백로그와 스프린트 진행 상황을 공유
코드 관리	main/develop/feature 브랜치 전략, Pull Request 기반 상호 코드 리뷰 후 병합
커뮤니케이션	상시 메신저 채널 + 주 1회 정기 회의, 결정 사항은 문서로 기록
문서 관리	설계·API·프롬프트 문서를 저장소에 함께 관리해 산출물 누락 방지
지식 공유	새로 학습한 기술(LLM·RAG 등)은 회의 시 짧게 공유해 특정 팀원 종속을 방지

7. 관련 연구 및 문헌 조사

7.1 관련 문헌 및 선행연구 조사

가. LLM 에이전트와 도구 사용

언어모델이 외부 도구를 호출해 행동하는 에이전트 패러다임은 추론과 행동을 결합한 ReAct(Yao et al., 2022)와 모델 스스로 도구 사용을 학습하는 Toolformer(Schick et al., 2023) 이후 표준 설계로 자리 잡았다. 본 과제의 대화형 에이전트(F9)와 일정 조작 기능은 Function Calling 기반 도구 호출 구조로 이 흐름을 적용하며, 도구 실행 결과를 다시 모델에 전달해 자연어로 응답하는 순환 구조를 채택한다.

나. 검색 증강 생성(RAG)

외부 문서를 검색해 근거로 삼는 RAG(Lewis et al., 2020)는 모델이 학습하지 않은 정보를 다루면서 환각을 줄이고 출처를 제시할 수 있어, 강의자료처럼 개별 사용자마다 다른 문서를 다루는 교육용 질의 응답에 적합하다. 본 과제는 강의자료 요약·시험 예상 문제 생성(F4)에 이를 적용하고, 청크 메타데이터에 출처 페이지를 저장해 응답의 검증 가능성을 확보한다.

다. 구조화 정보 추출

LLM에 출력 스키마를 지정해 비정형 문서에서 JSON을 추출하는 구조화 출력 기법은, 양식이 고정된 문서에만 동작하는 전통적 규칙·템플릿 기반 추출과 달리 형식이 제각각인 문서에 강건하다. 기관·부서마다 서식이 다른 공지·과제 정보를 다루어야 하는 본 과제의 특성상 핵심 방법론이며(F1·F6·F11), 한국어 문서에 대한 실제 추출 품질을 자체 테스트셋으로 정량 검증한다.

라. 일정 관리·학습 행동 연구

마감 관리와 미루기(procrastination) 관련 연구들은 마감을 인지하는 시점이 빠를수록, 그리고 과제가 작은 단위로 분해되어 일정에 배치될수록 완수율이 높아진다고 보고한다. 이는 마감을 미리 알려주는 브리핑(F10), 우선순위 제시(F7), 학습 블록 자동 배치(F8·F5)의 설계 근거가 된다. 또한 알림이 과도할 경우 무시되는 알림 피로(notification fatigue) 문제가 보고되므로, 본 과제는 "모든 정보를 알리는" 대신 "지금 필요한 것만 선별해 알리는" 방향으로 알림 정책을 설계한다.

7.2 기술 및 시장동향 조사

가. AI 비서 서비스 동향

- 범용 챗봇을 넘어 캘린더·이메일·문서와 연동되는 개인 비서형 AI 서비스가 빠르게 확산되고 있으나, 대부분 영어권·직장인 업무 중심으로 설계되어 한국 대학생의 학사 도메인(학사공지·LMS·장학 제도)에 특화된 서비스는 공백에 가깝다.

- LLM 의 도구 호출·구조화 출력 기능이 표준화되면서, 대화형 인터페이스에 실제 행동(캘린더 등록, 신청서 작성)을 결합한 에이전트형 서비스 구현 장벽이 크게 낮아졌다.

나. 대학생 대상 서비스 시장

- 시간표 관리·커뮤니티(에브리타임 등), 학점 계산기 등 개별 기능 앱은 널리 사용되고 있으나, 대부분 사용자가 정보를 직접 입력해야 하는 수동형 도구이다.
- 문서를 이해해 일정을 자동 생성하고 우선순위를 판단해 먼저 알려주는 능동형 에이전트는 아직 일반화되지 않아, "입력 부담 제거"라는 축에서 차별화 여지가 크다.

다. 국산 LLM 생태계

- Upstage Solar, LG EXAONE 등 한국어 특화 모델이 API 또는 오픈 가중치 형태로 제공되고, 문서 파싱(OCR)·임베딩까지 상용 제품군이 갖춰지면서 한국어 문서 중심 서비스를 국산 스택만으로 구성하는 것이 실용적인 선택지가 되었다.
- 경량 모델의 성능 향상과 단가 인하로, 학생 팀도 대량 문서 처리형 서비스를 현실적인 비용으로 운영할 수 있는 환경이 조성되었다. 본 과제는 이러한 환경에서의 실증 사례를 목표로 한다.

라. 자체 시장 조사 계획

과제 초기(7~8월)에 교내 대학생을 대상으로 설문(표본 30인 이상)을 실시해 일정 관리 실태, 마감 누락 경험, 기능별 선호도를 조사하고, 그 결과를 기능 개발 우선순위 결정과 사용성 평가 기준에 반영한다. 조사 결과는 결과보고서에 수록한다.

8. 결론

8.1 프로젝트 요약

본 프로젝트는 흩어진 학사 정보를 스스로 읽고, 일정으로 변환하고, 우선순위를 판단해 매일 능동적으로 알려주는 대학생 학사·일정 개인 비서 AI 에이전트 「유니버스(Univ-Us)」를 개발한다. 학사 일정 자동 등록, 졸업요건 트래커, 출결 경고, 강의자료 요약·시험 예상 문제 생성, 시험 공부 일정 추천, 과제 마감 자동 등록과 우선순위 배치, 공강 기반 학습 플랜, 자연어 일정 조작, 아침 브리핑, 장학 공지 매칭·신청서 자동 작성, 대외활동·사업단 공지 알림, 팀플 일정 조율, 사업단 프로젝트 관리 플랫폼까지 15개 기능을 핵심 범위로 구현한다.

기술적으로는 한국어 특화 국산 LLM(Upstage Solar Pro 3)을 주 모델로 삼고, 문서 파싱(Document Parse)과 임베딩(Solar Embedding)을 결합한 실용 파이프라인을 Next.js·FastAPI·PostgreSQL(pgvector) 기반 웹 서비스로 구현한다. 판단이 필요한 작업은 LLM에, 정확성이 요구되는 계산은 규칙 기반 코드에 맡기는 역할 분리와, 실제 행동 전 사용자 확인을 거치는 Human-in-the-loop 설계로 신뢰성을 확보한다.

8.2 기대효과

- 사용자 측면: 대학생의 일정 관리 부담과 마감 누락으로 인한 실질적 손해(과제 미제출, 장학·프로그램 신청 기회 상실, 결석 초과)를 줄이고, 정보 탐색에 쓰이는 시간을 학습에 재배분할 수 있게 한다.
- 교육 측면: 팀원들이 문서 이해, RAG, 도구 호출, 스케줄링, 외부 API 연동 등 LLM 에이전트 개발의 전 과정을 실전으로 경험하고, 기업 멘토링을 통해 산업 현장의 설계·품질 기준을 학습한다.
- 기술 측면: 한국어 학사 문서에 대한 국산 LLM의 구조화 추출 성능을 자체 테스트셋으로 정량 검증함으로써, 한국어 문서 기반 AI 서비스 설계에 참고 가능한 실증 데이터를 남긴다.
- 확장 측면: 검증된 "문서 이해 → 일정 변환 → 능동 브리핑 → 승인 워크플로" 구조는 사업단 프로젝트 관리 플랫폼(F17)을 비롯해 직장인 업무 비서, 기관 행정 지원 등 인접 도메인으로 이전 가능한 자산이 된다.

8.3 향후 계획

본 과제 기간 내에는 위 15개 기능의 구현과 검증에 집중하고, 종료 이후에는 교내 시범 서비스 운영을 통한 사용자 확대, LMS 공식 연동 협의, 모바일 알림 채널(카카오톡 등) 확대, 타 대학 학사 체계로의 일반화를 후속 과제로 검토한다.

APPENDIX. 참고문헌 및 정보사이트

가. 참고문헌

- Yao, S. et al. (2022). ReAct: Synergizing Reasoning and Acting in Language Models. arXiv:2210.03629.
- Lewis, P. et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 2020.
- Schick, T. et al. (2023). Toolformer: Language Models Can Teach Themselves to Use Tools. arXiv:2302.04761.
- Gao, Y. et al. (2023). Retrieval-Augmented Generation for Large Language Models: A Survey. arXiv:2312.10997.

나. 기술 문서·정보사이트

구분	주소
Upstage Solar API 문서	https://console.upstage.ai/docs
Upstage Document Parse	https://www.upstage.ai/products/document-parse
LangChain 공식 문서	https://python.langchain.com
FastAPI 공식 문서	https://fastapi.tiangolo.com
Next.js 공식 문서	https://nextjs.org/docs
PostgreSQL / pgvector	https://www.postgresql.org/docs , https://github.com/pgvector/pgvector
Google Calendar API	https://developers.google.com/calendar
Gmail API	https://developers.google.com/gmail/api
APScheduler	https://apscheduler.readthedocs.io
전남대학교 인공지능혁신융합대학사업단	https://aicoss.kr/www